



Mapping

V Muthukumar

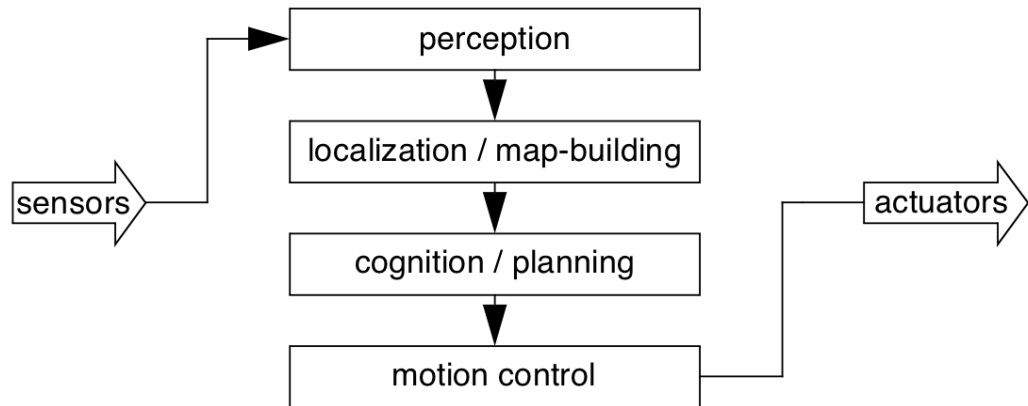
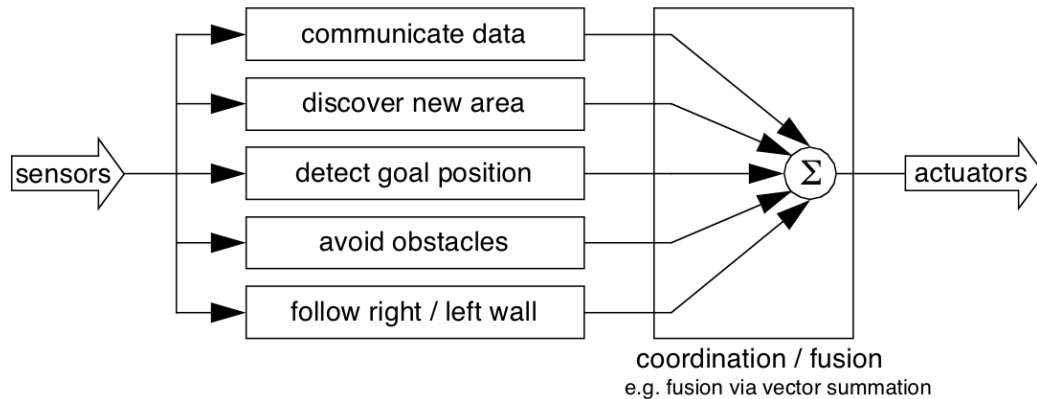
Mapping: inferring a map given **locations**





Architectures for Navigation

An Architecture for Behavior-based Navigation



An Architecture for Map-based (or model-based) Navigation





Mapping

- Path-planning/Navigation - requires a map of the environment and the robot to be aware of its location with respect to the map
- Localization - determine the robot pose from landmarks position and encoder-angular measurements.
- Mapping – needed to efficiently determine location and navigation.
- Given the (command data/cmd_vel) input data u_i and (odometry/sensor data) perception data z_i .

$$d = \{u_1, z_1, u_2, z_2, \dots, u_n, z_n\}$$

- Mapping problem is to calculate the most likely map

$$m^* = \arg \max_m P(m \mid d)$$





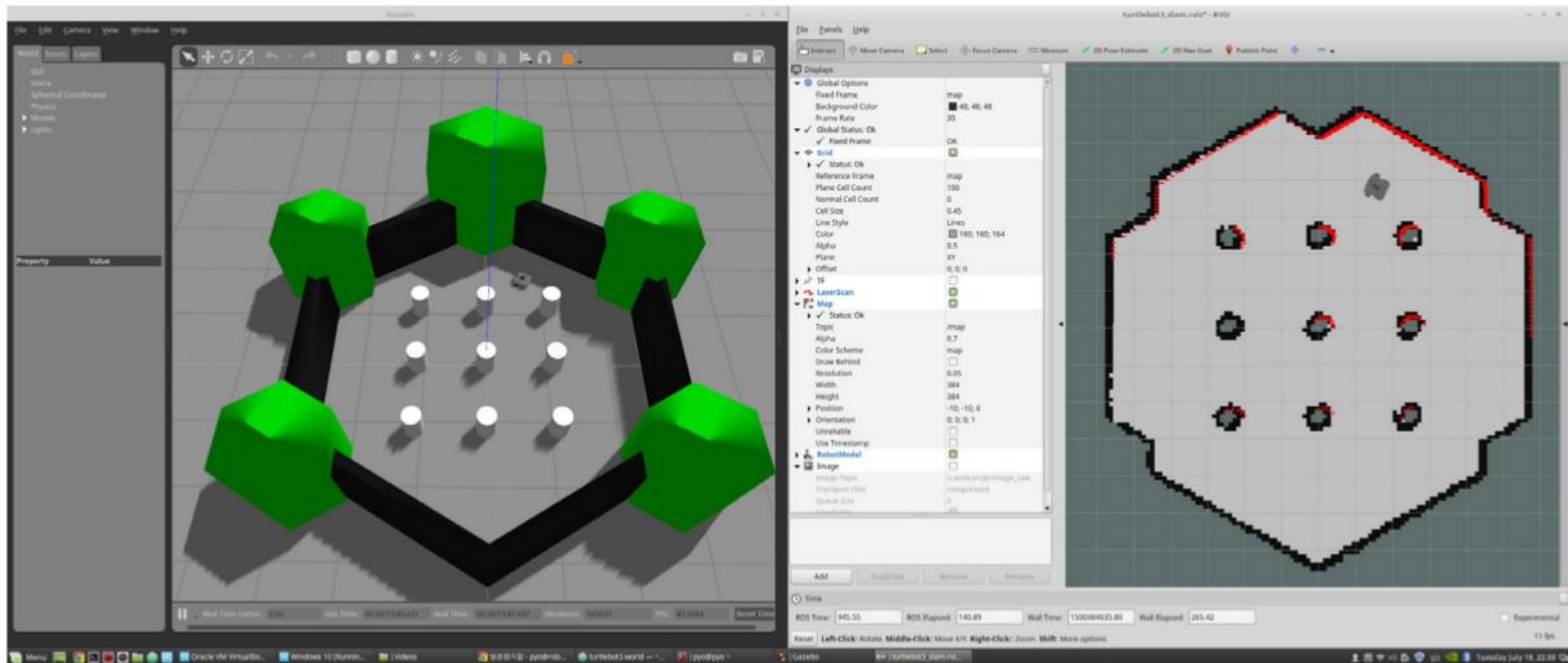
How are maps built?

- Autonomous Map Building
 - Starting from an arbitrary initial point, a mobile robot should be able to autonomously explore the environment with its on-board sensors, gain knowledge about it, interpret the scene, build an appropriate map and localize itself relative to this map.
- Basic Requirements of a Map:
 - a way to incorporate newly sensed information into the existing world model
 - information and procedures for estimating the robot's position
 - information to do path planning and other navigation task (e.g. obstacle avoidance)





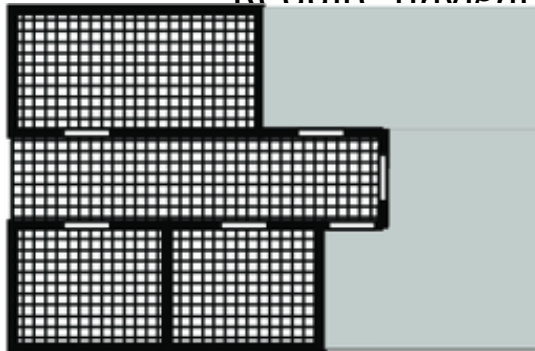
What are we looking for?



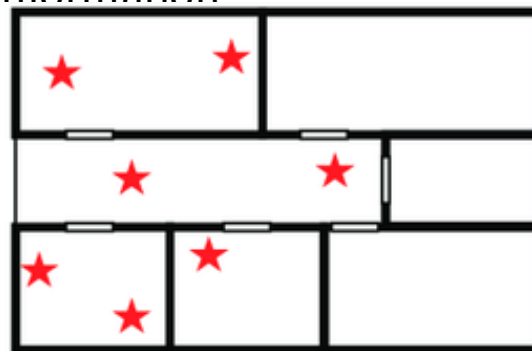


Mapping Overview

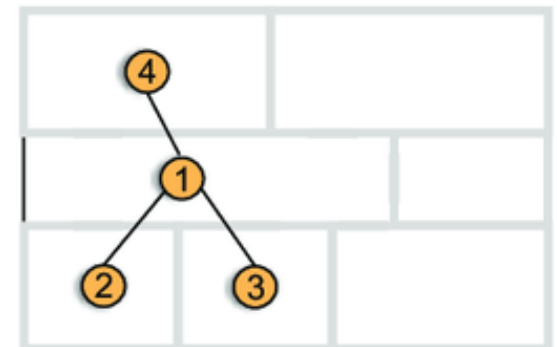
- Metric Mapping
 - Geometric representations
 - Occupancy Grids
 - Larger maps much more computationally intensive
- Feature based Maps
- Topological Mapping
 - Milestones with connections
 - Require navigation information



(a)



(b)



(c)





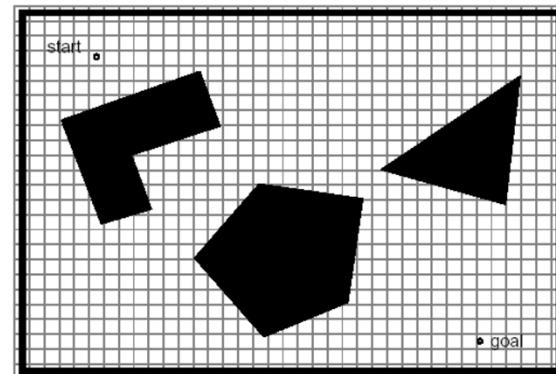
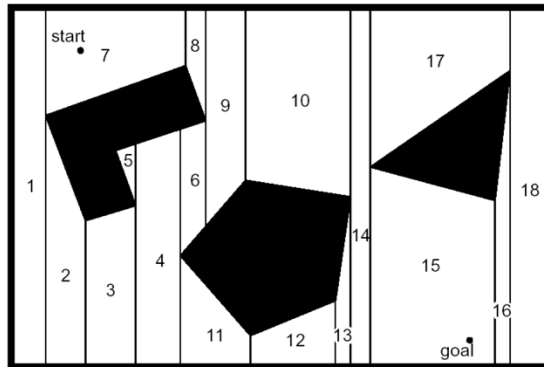
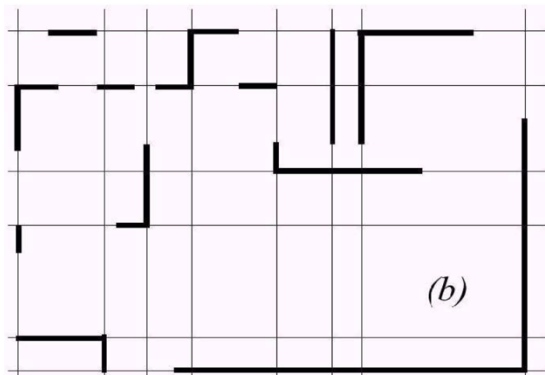
Map Representation

- Metric/Discrete (Grid) & Continuous (Topological), and Feature based (landmark)*
- Discrete (Grid)
 - map is sub-divided into chunks of equal (e.g., a grid or hexagonal map)
- Topological maps
 - differing sizes
 - continuous Representation
- Both requires the definition of inner (obstacles) and outer boundaries, typically in the form of a polygon, whereas paths can be encoded as sequences of points defined by real numbers.
 - Obstacle Maps
 - Occupancy Grid Map.

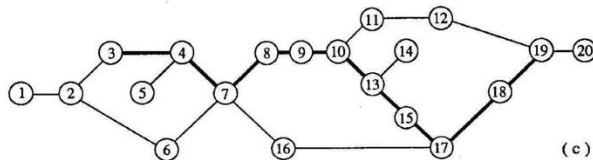




Maps and Representations

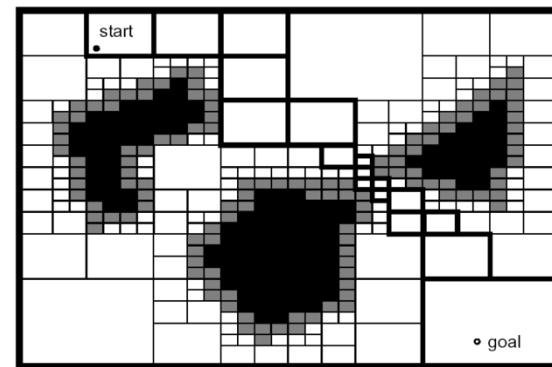
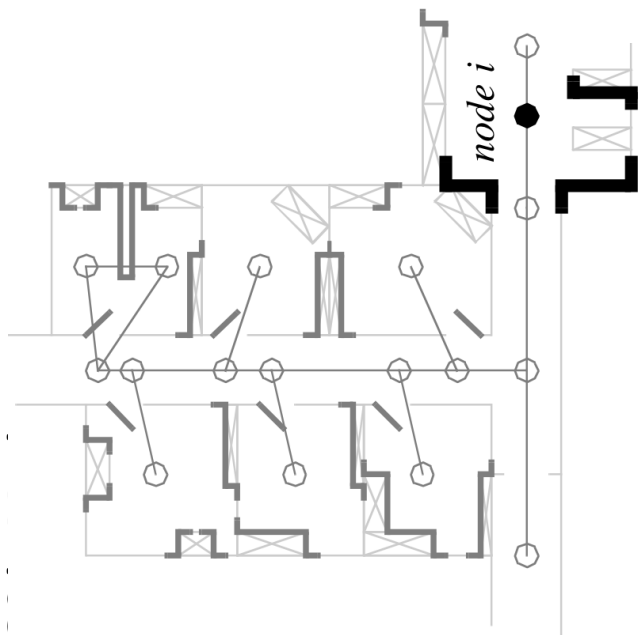


Grid



(c)

Topological



Quadtree





Sensors for Mapping

- Contact sensors: Bumpers
- Internal sensors
 - Accelerometers (spring-mounted masses)
 - Gyroscopes (spinning mass, laser light)
 - Compasses, inclinometers (earth magnetic field, gravity)
- Proximity sensors
 - Sonar (time of flight)
 - Radar (phase and frequency)
 - Laser range-finders (triangulation, tof, phase)
 - Infrared (intensity)
- Visual sensors: Cameras (Monocular, Stereo, Depth, LIDAR)
- Satellite-based sensors: GPS





Scan Matching Overview

- Problem statement:
 - Given a scan and a map, or a scan and a scan, or a map and a map, find the rigid-body transformation (translation + rotation) that aligns them best.
- Scan matching is a technique for aligning the sensor readings.
- Map matching when aligning scan with existing map.
- likelihood field based, targeting on optimizing measurement probability $P(z|x,m)$.
 - comparing the measurements of your scan with your previously computed map
- ICP (Iterative Closest Point)-based
 - powerful algorithm for calculating the displacement between scans
https://wiki.ros.org/laser_scan_matcher

http://wiki.ros.org/multirobot_map_merge





Iterated Closest Points (ICP)

- Given: two corresponding point sets:

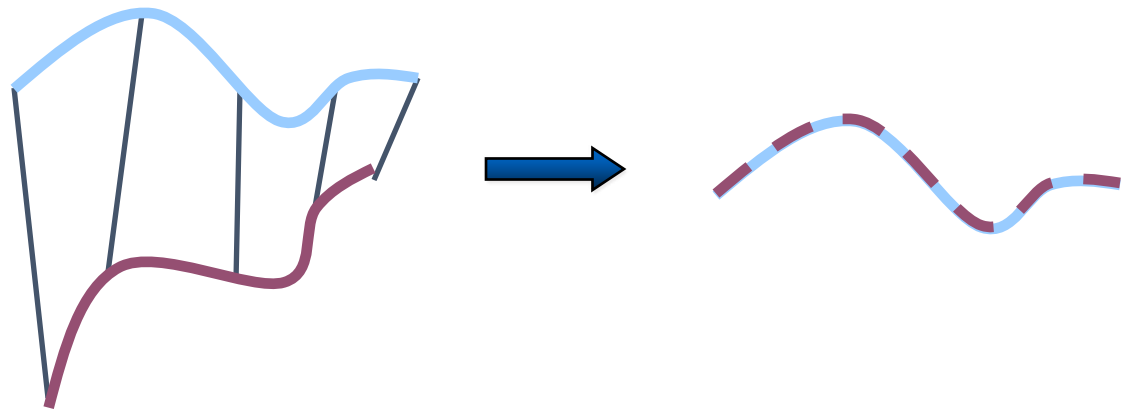
$$X = \{x_1, \dots, x_n\}$$

$$P = \{p_1, \dots, p_n\}$$

- Wanted: translation t and rotation R that minimizes the sum of the squared error:

$$E(R, t) = \frac{1}{N_p} \sum_{i=1}^{N_p} \|x_i - Rp_i - t\|^2$$

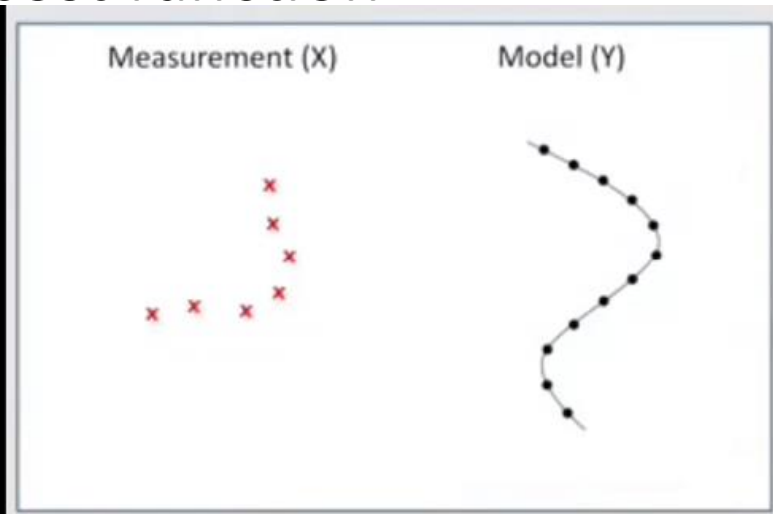
If the correct correspondences are known, the correct relative rotation/translation can be calculated in closed form.





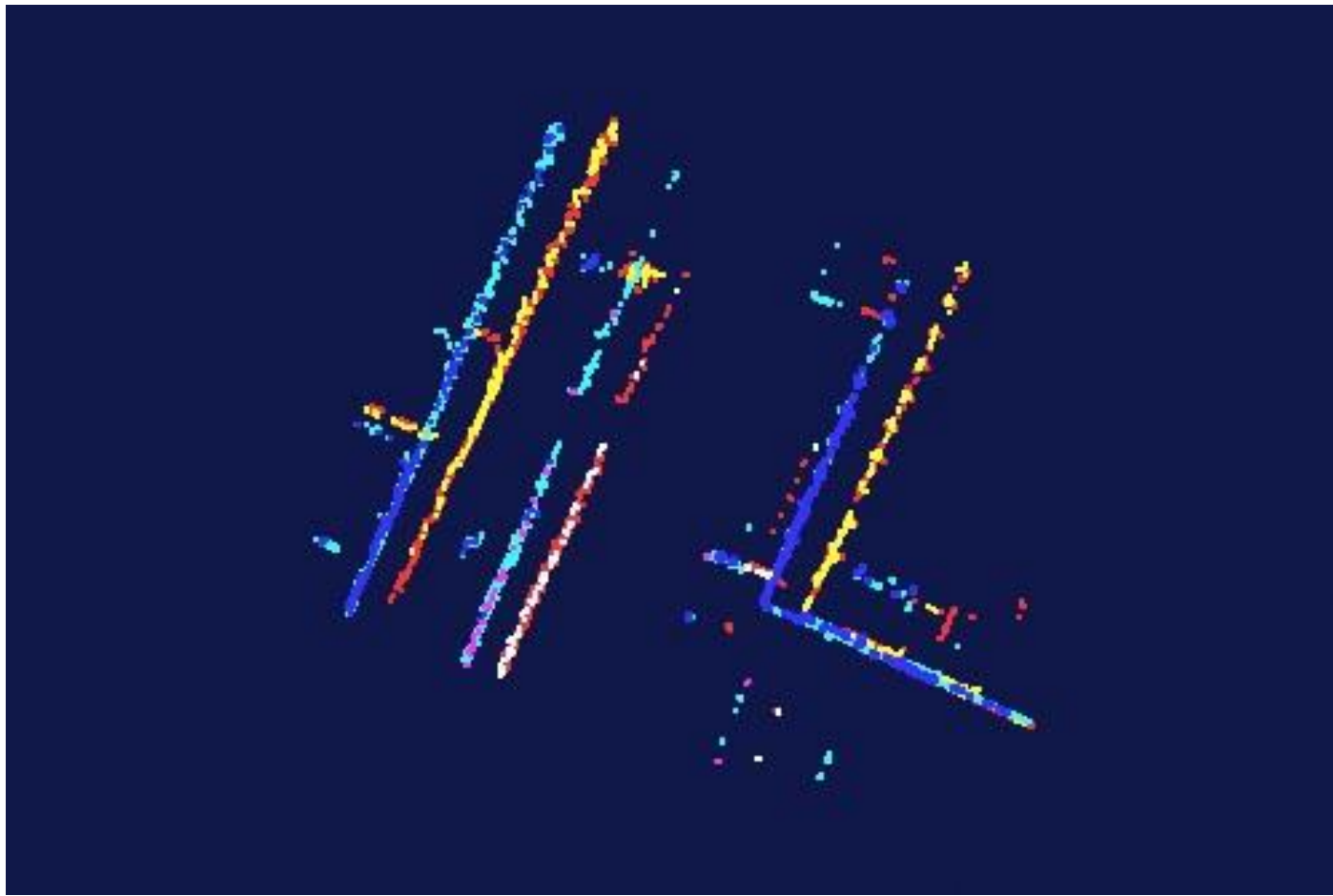
ICP Algorithm

- Data association, find the corresponding points between target frame and reference frame
 - Closest point
 - Normal shooting
 - Closest compatible point
 - Projection
- Calculate the rigid-body transformation T from corresponding points using cost function
- Transform the target frame transformation
- Iterate until it converge





IPC Example





ICP-Variants

- Variants on the following stages of ICP have been proposed:
 - Point subsets (from one or both point sets)
 - Weighting the correspondences
 - Data association
 - Rejecting certain (outlier) point pairs
- In ROS
 - **laser_scan_matcher** for scan matching with ICP
 - **PCL** libraries





Mapping Problems

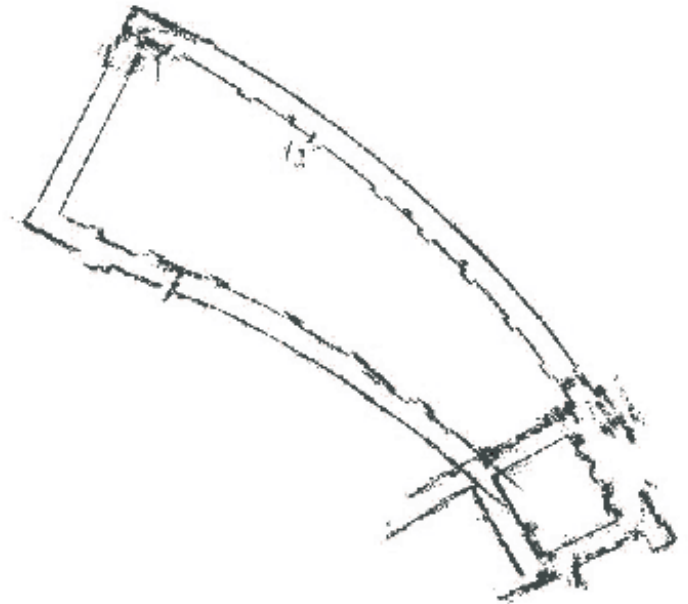
- Measurement noise
 - Sensor and Position noise is not independent
- Map size
 - High resolution maps can be very large
- Correspondence
 - Do multiple measurements at different times correspond to the same object?
- Dynamic environments
 - Most current algorithms assume a static environment





Errors in Mapping

- Odometry Errors: heading and distance measurements accumulate errors with time





Current State of Mapping

- Algorithms
 - Robust for static, structured, and limited-size environments
 - Probabilistic
 - Correspondence Problem
 - Incremental vs. Multi-pass
- Unsolved Areas
 - Dynamic environments
 - Planning exploration paths of unknown environments





ROS Mapping Packages

1. **gmapping**: gmapping (Grid-based FastSLAM) is a popular package for generating 2D occupancy grid maps. It uses the FastSLAM algorithm to build a probabilistic map of the environment based on laser range data and odometry information.
2. **Cartographer**: Cartographer is another package that can create 2D and 3D maps. It uses various sensor inputs, such as LIDAR and IMU, to perform simultaneous localization and mapping (SLAM). Cartographer is known for its accurate mapping capabilities.
3. **Hector SLAM**: Hector SLAM is a lightweight mapping package suitable for 2D mapping using LIDAR data. It is designed to work well in real-time applications and can be used on robots with limited computational resources.
4. **RTAB-Map**: RTAB-Map (Real-Time Appearance-Based Mapping) is a versatile package that not only creates maps but also performs loop closure detection for better mapping accuracy and supports 3D mapping as well.
5. **Karto SLAM**: Karto SLAM is a feature-based mapping package that uses scan-matching algorithms to build 2D maps. It is known for its fast mapping capabilities.
6. **OctoMap**: OctoMap is primarily used for 3D mapping. It generates an Octree-based representation of the environment, which provides a more detailed and volumetric map.





Demos

- Mobile Robot Programming Toolkit (**MRPT**) SLAM
 - http://wiki.ros.org/mrpt_slam
 - MRPT's 2D ICP-SLAM algorithms
 - http://wiki.ros.org/mrpt_icp_slam_2d
- Gmapping
 - <http://wiki.ros.org/gmapping>
 - ICP built-in
 - RBPF (Rao-Blackwellized Particle Filter - Primarily used for SLAM, handles both pose estimation and map creation.
- Hector SLAM
 - http://wiki.ros.org/hector_slam

